Folding constant learners in NGBoost: bit-identical predictions, faster inference

Moussa Ouallaf

Abstract

NGBoost, boosting that predicts a full probability distribution, calls every stored tree on every prediction, even a tree that returns the same value for every input. We fold such constant trees into a precomputed start value in the original floating-point order, and walk the other trees after one input check per call instead of one per tree. Against pristine ngboost 0.5.11 on three public data sets (3 seeded splits each), one-row prediction on tuned models was 3.67× faster on concrete (paired log ratio -1.300 ± 0.005 SE over seeds), 3.60× on yacht and 3.51× on credit-g, with bit-identical outputs on all 6,994 test rows of 90 fitted models. The tuned models held no constant tree, so this gain is the single input check alone (folding on top: a tie, -0.007 ± 0.005). Folding paid off when every tree was constant, which in our runs happened only on synthetic data without signal: 1,808× to 2,215× faster. We prove when a model must be all-constant and turn that bound into a fit-time warning.

1 Problem

NGBoost [Duan et al., 2020] fits one regression tree per distribution parameter per boosting round, following the natural gradient (the gradient rescaled to the distribution's geometry, Amari, 1998). Prediction sums all those trees:

$$\theta_k(x) \;=\; \theta_{0,k} \;-\; \sum_{i=1}^{M} \big(\eta\, f_{ik}(x_{c_i})\big)\, \rho_i, \qquad k = 1, \ldots, P. \tag{1}$$

Symbol	Meaning	Symbol	Meaning
$\theta_k(x)$	parameter k of the predicted distribution	η	learning rate
$\theta_{0,k}$	initial value of parameter k	ρ_i	line-search scale of round i
f_{ik}	tree of round i, parameter k	c_i	feature columns sampled in round i
M, P	number of rounds, number of parameters		

The upstream library [Stanford ML Group, 2025] evaluates Equation (1) as a loop that calls scikit-learn's `predict` [Pedregosa et al., 2011] MP times. Each call re-validates the input (array checks, a float32 cast, a feature count). For a single row this validation costs more than walking the tree, and we measure it as the main cost in Section 4.

The loop also pays for trees that do nothing, and this is the case we set out to remove. A tree with one node, a root-only tree, returns the same value for every input, but it still costs a full `predict` call. This happens when each round's minibatch is smaller than twice the minimum leaf size, and a tuning search can land there without any error. The model then predicts one fixed distribution for every input, yet one prediction still runs MP calls.

Preprint.

2 The change

We change how Equation (1) is evaluated, not what it computes. Write $a \ominus b$ and $a \otimes b$ for float64 subtraction and multiplication with IEEE rounding [Goldberg, 1991]. Upstream runs, for each column k and round i in order,

$$\theta_k^{(i)} = \theta_k^{(i-1)} \ominus \big((\eta \otimes f_{ik}(x_{c_i})) \otimes \rho_i\big), \qquad \theta_k^{(0)} = \theta_{0,k}. \tag{2}$$

Folding. Call learner (i, k) constant when $f_{ik}(x) = v_{ik}$ for every input the model accepts: a one-node tree, a tree whose leaves all hold the same value, or a `DummyRegressor`. Its update is then the scalar $s_{ik} = (\eta \otimes v_{ik}) \otimes \rho_i$. Let i_k^* be the first round in which column k has an input-dependent learner. The fold plan stores

$$\text{start}_k = \big(\cdots\big((\theta_{0,k} \ominus s_{1k}) \ominus s_{2k}\big)\cdots\big) \ominus s_{i_k^*-1,\,k}, \tag{3}$$

and, from round i_k^* on, an ordered list of tree steps (Equation (2)) and scalar steps $\theta_k \leftarrow \theta_k \ominus s_{ik}$. We never merge scalar steps after the first split, because merging would reorder floating-point subtractions and change the last bit. Prediction broadcasts start to all rows and walks the list, so the number of learner calls falls from MP to the number of input-dependent learners N_s; with $N_s = 0$ prediction is one broadcast.

Direct traversal. We validate the input and cast it to float32 once per call, and then walk each remaining tree with scikit-learn's low-level `tree_.predict` with input checks off. Upstream `predict` performs the same cast and the same walk, so the values are equal.

Lemma 1 (Folding is bit-identical)**.** *For every fitted model and every row x that upstream accepts, folded prediction with direct traversal returns the same float64 bits as Equation* (2) *after round M, and it rejects exactly the rows that upstream rejects.*

Lemma 2 (When no tree can split)**.** *Let each round fit its trees on $m = \lfloor \phi\, n' \rfloor$ rows, with minibatch fraction ϕ and n' training rows after any early-stopping hold-out. If $m < \max(\textit{min_samples_split}, 2L)$, with L the minimum leaf size, every learner is a one-node tree and $\theta(x) =$ start for every x: the model is one fixed distribution.*

Both proofs are in Appendix A. For the rejection part, the plan probes every skipped learner once on a zero, a NaN and an infinity row. We turn Lemma 2 into a fit-time warning that names the bound, so a user learns that the tuned model ignores its input. The upstream-ready patch for ngboost 0.5.11 is 393 diff lines with tests; rounds stay stored, so staged prediction and `partial_fit` are unchanged. Nothing here depends on the data: the method applies to any fitted NGBoost model, and Appendix D gives the four calls and the recipe for other additive tree ensembles.

3 Setup

Data. Concrete [Yeh, 1998] (1,030 rows, 8 features) and yacht (308 rows, 6 features) come from the UCI collection [Dua and Graff, 2019]. They are Normal regressions with the test masks of NGBoost's own benchmark [Hernández-Lobato and Adams, 2015, Duan et al., 2020]. Credit-g comes from OpenML [Vanschoren et al., 2014] (1,000 rows, 61 encoded features) as a Bernoulli classification. Each data set has 3 splits, split s fitted with seed s; no model is fitted on test rows. The search sees only split 0's training rows, which may hold test rows of splits 1 and 2; this can flatter their held-out NLL but not equality or latency.

Models. A random search (30 trials on concrete, 20 on yacht and credit-g, 3-fold cross-validated negative log-likelihood (NLL, lower is a better fit)) picks depth, leaf size, minibatch fraction, learning rate and the number of rounds on split 0's training rows. We also fit ngboost's default tree (depth 3, leaf 1) and a grid of leaf sizes {1, 20, 100, 400} $\times$ minibatch fractions {0.2, 0.5, 1.0} $\times$ depths {1, 3} with 500 rounds.

Arms. All arms predict from the same fitted model, so data, features and tuning are equal by construction: pristine ngboost 0.5.11 (the baseline), fold only, direct traversal only, ours (fold plus direct traversal), and an inexact ablation that merges all scalar steps of a column.

Metrics. Equality is checked bit for bit against upstream `pred_param` on every test row of every fitted model. For latency each arm is first called once untimed (warm caches), then the arms run

interleaved in a rotating order, each call timed with `perf_counter` on one thread of an Intel Xeon E5-1410 v2 (2.8 GHz), 100 repetitions for one row and 50 for 100 rows. Per repetition r the paired statistic is

$$d_r = \log t_r^{\text{ours}} - \log t_r^{\text{arm}}, \qquad \text{speedup} = \exp(-\bar{d}), \tag{4}$$

where $\bar{d}$ is the mean over seeds of the per-seed median of d_r; negative d means ours is faster. The benchmark SE is the standard deviation of the per-seed medians over $\sqrt{3}$. We add a two-level bootstrap [Lahiri, 2003] that resamples seeds, then moving blocks of 10 consecutive repetitions (consecutive timings share machine load), with 2,000 resamples, and a difference inside its SE is called a tie.

Variants tried. We ran 4 arms against upstream and 3 follow-up settings. The arms, the grid and the tuned and default configurations were fixed in a written plan before any run. The follow-ups (small data, low signal, one-SE model selection) were added after the tuned models came out with no constant learners, so they are post hoc.

4 Results

Equality holds everywhere. Fold only, direct traversal only and ours returned the same bits as upstream on all 6,994 test rows of 90 fitted models (Table 1, last row). The merged ablation did not: only 73 to 80% of rows matched bitwise, with a largest difference of 4.0×10^{-15}.

Tuned models had no constant learners. The constant share (the fraction of a model's learners that are constant) was 0% on all 3 seeds of all 3 data sets, and 0% at the default configuration. Table 1 therefore measures direct traversal, not folding. Ours was 3.51× to 3.67× faster than upstream for one row, and ours against direct traversal only was a tie on every data set: the largest difference, concrete -0.007 ± 0.005, is 1.01×, inside the two-level SE of 0.009 with a 0.99–1.03× interval; with nothing to remove, folding adds nothing.

Table 1: On tuned models the whole one-row gain over upstream comes from direct traversal; ours ties direct traversal only. Cells give the speedup of ours over the arm and the paired log ratio $\bar{d} \pm$ benchmark SE ($n = 3$ seeds); the 95% interval is the two-level bootstrap. Below 1 means the arm is faster.

Arm	concrete	yacht	credit-g
upstream ngboost 0.5.11	3.67× (-1.300 ± 0.005)	3.60× (-1.280 ± 0.017)	3.51× (-1.254 ± 0.004)
95% interval	3.61–3.80	3.48–3.69	3.47–3.53
fold only	3.83× (-1.344 ± 0.005)	3.78× (-1.330 ± 0.009)	3.52× (-1.259 ± 0.010)
direct traversal only	1.01× (-0.007 ± 0.005)	1.01× (-0.005 ± 0.004)	1.00× (0.000 ± 0.002)
merged steps (inexact)	0.97× (0.035 ± 0.004)	0.96× (0.039 ± 0.002)	0.95× (0.052 ± 0.003)
p50 ms, upstream / ours	91.4 / 23.8	36.0 / 9.7	70.4 / 20.3
bitwise rows (models)	3,090 (30)	904 (30)	3,000 (30)

Figure 1 adds the per-seed medians. The gain over upstream holds at larger batches: 3.94× on concrete and 3.57× on credit-g at 100 rows, 3.86× on yacht's 30-row test set. Fold only is 4% (concrete) and 5% (yacht) slower than upstream, the cost of its per-round bookkeeping with no learner removed. The merged ablation is faster than ours by the same margin (ours runs at 0.95 to 0.97 of its speed): it does the same learner work, so the gap is loop overhead in our code, not the price of exactness.

Constant learners appear exactly where Lemma 2 says. On the grid the constant share was never partial: 22 of 72 points were all-constant (concrete 6, yacht 10, credit-g 6) (Table 2, Appendix C), and these were precisely the points where the minibatch was below the split bound (Figure 2). There one-row prediction was 4,742× to 10,069× faster on yacht, 4,897× to 8,153× on concrete and 964× to 1,842× on credit-g. Two yacht fits that upstream stopped after one round gained only 7.00× to 7.81×, because the gain scales with the number of rounds removed. The search never picks these points because they fit worse: held-out NLL 4.23 against 2.88 for the best concrete point, 2.07 against -0.80 on yacht.

When does the search pick an all-constant model? In our runs, only without usable signal. On Friedman's first benchmark function [Friedman, 1991] with the signal scaled down to none and to a

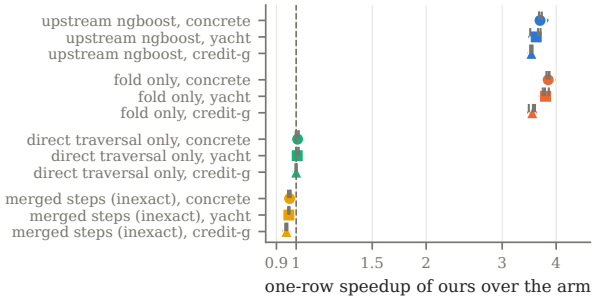


Figure 1: Ours beats upstream 3.51× to 3.67× for one row and ties direct traversal only, because tuned models hold no constant learner. Dots: speedup at $\bar{d}$; bars: two-level bootstrap 95% interval; ticks: per-seed medians; log axis.

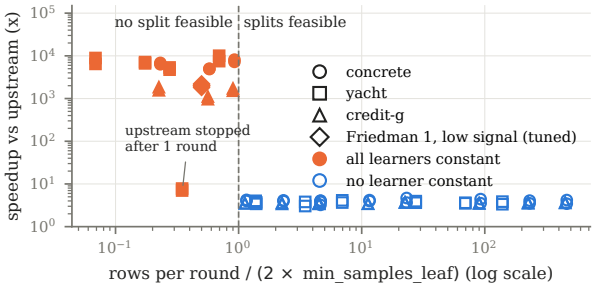


Figure 2: The split bound separates the two regimes: left of the line every learner is constant and ours gains 964× to 10,069×; right of it no learner is constant and only direct traversal helps. One point per grid fit (one seed) and per low-signal fit.

small fraction of the noise (400 training rows, 15 trials), the search picked leaf 200 with minibatch 0.5: 200 rows per round where 400 are needed to split. Every learner was constant in all 6 fits, and one-row prediction dropped from 320–394 ms upstream to 0.18 ms. That is 1,808× to 2,215× per fit, bit-identical, with a block-bootstrap SE of d between 0.02 and 0.07 over 60 repetitions. Fold only was faster still, 0.16 ms, because the float32 cast of direct traversal is pure overhead there. Small data alone did not get there: credit-g cut to 100 training rows picked 3 rounds of depth-4 trees, constant share 0%. The one-SE rule [Breiman et al., 1984], which picks the simplest model within one standard error of the best cross-validated score, did pick all-constant models, at a held-out NLL tied with the best pick: +0.020 ± 0.014 nats on credit-g ($n = 100$ test rows), −0.00012 ± 0.00016 and −0.00011 ± 0.00012 on the two Friedman levels ($n = 200$).

5 Limits

Folding gave no speedup on any tuned public model, and we tested only three data sets, so we do not claim how often real tuned models are all-constant (Appendix B reports one case outside the benchmark). The gain there is direct traversal, which is plain engineering on scikit-learn's call path. The large folding gains need a model with no usable signal, and such a model is better reported to the user than served faster, so we think the warning from Lemma 2 is the more useful part.

The run is smaller than we planned: 3 data sets instead of the UCI suite, 3 seeds, a 3-fold search, and no external speed reference such as compiled tree libraries [Cho and Li, 2018, Nakandala et al., 2020]. Building the plan takes 16%, 18% and 29% of fit time on concrete, yacht and credit-g (median). The host was shared (load average 7 to 20), and interleaving protects the ratios, not the absolute milliseconds. The benchmark SE rests on 3 seeds and is smaller than the two-level SE (concrete 0.005 against 0.013); we report both. Folding linear learners with zero coefficients is exact only for finite inputs; it is opt-in and we did not benchmark it.

References

Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998. doi: 10.1162/089976698300017746.

Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, Belmont, CA, 1984.

Hyunsu Cho and Mu Li. Treelite: Toolbox for decision tree deployment. In *Proceedings of the Conference on Systems and Machine Learning (SysML)*, 2018.

Dheeru Dua and Casey Graff. UCI machine learning repository. `http://archive.ics.uci.edu/ml`, 2019.

Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. NGBoost: Natural gradient boosting for probabilistic prediction. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 2690–2700, 2020.

Jerome H. Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 19(1):1–67, 1991. doi: 10.1214/aos/1176347963.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. doi: 10.1145/103162.103163.

José Miguel Hernández-Lobato and Ryan P. Adams. Probabilistic backpropagation for scalable learning of Bayesian neural networks. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1861–1869, 2015.

S. N. Lahiri. *Resampling Methods for Dependent Data*. Springer, New York, 2003. doi: 10.1007/978-1-4757-3803-2.

Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. A tensor compiler for unified machine learning prediction serving. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 899–917, 2020.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Stanford ML Group. ngboost: Natural gradient boosting for probabilistic prediction, version 0.5.11. `https://github.com/stanfordmlgroup/ngboost`, 2025. Apache-2.0 license.

Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2):49–60, 2014. doi: 10.1145/2641190.2641198.

I-Cheng Yeh. Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12):1797–1808, 1998. doi: 10.1016/S0008-8846(98)00165-3.

A Proofs

Proof of Lemma 1. **Given:** 1) the upstream update, Equation (2); 2) learner (i, k) is constant: $f_{ik}(x) = v_{ik}$ for every row x the model accepts; 3) IEEE 754 arithmetic: the result of $\ominus$ and $\otimes$ depends only on the two operands [Goldberg, 1991]; 4) $1 \otimes a = a$ for every finite float a (folded prediction broadcasts start_k to all rows by multiplying it with a column of ones); 5) scikit-learn's input check rejects a row only for a wrong column count or a non-finite entry (NaN, or infinity, including a finite value that overflows on the float32 cast).

Choose: a column k, a row x that upstream accepts, and the chain $a_0 = \theta_{0,k}$, $a_i = a_{i-1} \ominus s_{ik}$ for $i < i_k^*$, so $a_{i_k^*-1} = \text{start}_k$ (Equation (3)).

Need to prove: $\theta_k^{(i)}(x) = a_i$ for all $i < i_k^*$. Assume it for $i-1$ (it holds for $i = 0$ by definition). Then

$$\begin{aligned}
\theta_k^{(i)}(x) &= \theta_k^{(i-1)}(x) \ominus \big((\eta \otimes f_{ik}(x_{c_i})) \otimes \rho_i\big) && \text{given 1} \\
&= \theta_k^{(i-1)}(x) \ominus \big((\eta \otimes v_{ik}) \otimes \rho_i\big) && \text{given 2, round } i < i_k^* \text{ is constant} \\
&= a_{i-1} \ominus \big((\eta \otimes v_{ik}) \otimes \rho_i\big) && \text{induction assumption, given 3} \\
&= a_{i-1} \ominus s_{ik} = a_i && \text{definition of } s_{ik} \text{ and } a_i
\end{aligned}$$

Folded prediction starts at $1 \otimes \text{start}_k = \text{start}_k$ (given 4), which equals $\theta_k^{(i_k^*-1)}(x)$.

Same method, but on rounds $i \geq i_k^*$. Assume folded and upstream agree after round $i-1$. If (i,k) is a tree step, folded applies Equation (2) itself, with direct traversal in place of `predict`. If it is a scalar step,

$$\begin{aligned}
\theta_k^{(i-1)} \ominus s_{ik} &= \theta_k^{(i-1)} \ominus \big((\eta \otimes v_{ik}) \otimes \rho_i\big) && \text{definition of } s_{ik} \\
&= \theta_k^{(i-1)} \ominus \big((\eta \otimes f_{ik}(x_{c_i})) \otimes \rho_i\big) && \text{given 2}
\end{aligned}$$

which is the upstream update. Steps are kept in round order and never merged, so each side applies the same operation to the same operands (given 3).

Precondition: direct traversal returns the same tree output. Upstream `predict` casts x_{c_i} to float32 and walks the tree; direct traversal casts the same columns to float32 once and walks the same nodes with the same thresholds. Same comparisons give the same leaf, and the same leaf gives the same stored float64 value.

Precondition: given 2 holds on every accepted row. A one-node tree stops at the root for any x; a tree whose leaves all hold v_{ik} returns v_{ik} whichever leaf x reaches; a `DummyRegressor` ignores x.

Precondition: the same rows are rejected. By given 5 a learner can reject a row for only two reasons. The column count is checked once per call against the model, as every upstream call checks it. Whether a skipped learner rejects NaN and whether it rejects infinity is read from the probe on a NaN row and an infinity row, and folded prediction rejects exactly those inputs, after the same float32 cast when the learner casts. Learners that are still called reject what they rejected before.

Therefore, by induction over $i = 1, \ldots, M$, folded $\theta_k(x)$ equals upstream $\theta_k^{(M)}(x)$ bit for bit, for every k and every accepted x. □

Proof of Lemma 2. **Given:** 1) a CART node [Breiman et al., 1984] as fitted by scikit-learn splits only if it holds at least min_samples_split rows and each child gets at least L rows; 2) each round fits its trees on $m = \lfloor \phi\, n' \rfloor$ rows; 3) Lemma 1.

Choose: any round i and parameter k, and suppose the root of f_{ik} splits into children with m_ℓ and m_r rows.

Need to prove: a split requires $m \geq \max(\text{min_samples_split}, 2L)$.

$$\begin{aligned}
m &= m_\ell + m_r && \text{every row goes to one child} \\
&\geq L + L = 2L && \text{given 1, leaf condition} \\
m &\geq \text{min_samples_split} && \text{given 1, split condition}
\end{aligned}$$

So if $m < \max(\text{min_samples_split}, 2L)$ the root cannot split and f_{ik} is a one-node tree. m is the same in every round, so this holds for all i and k, and $i_k^* = M + 1$ for every column.

Therefore, by Lemma 1 with no tree step, $\theta_k(x) = \text{start}_k$ for every x: one fixed distribution. Under K-fold cross-validation n' is the training-fold size, so the bound bites earlier during tuning than on the full data. The converse is not claimed: a feasible split can still be skipped when no split lowers the loss. □

B One deployment outside the benchmark

On a proprietary binary forecasting task, a tuned NGBoost model (a seed ensemble of 8 models, three-parameter distribution) had every base learner constant because its tuned leaf size exceeded half the minibatch. Folding reduced single-row prediction for the ensemble from 1,338 ms to 29 ms p50 (46x; 5th-percentile per-call ratio 24.6x; a loaded 8-core host) with bit-identical outputs on every held-out row. On a tuned configuration whose trees split on two of three parameters (one third of learners constant), the gain was 1.37x, also bit-identical. This is one deployment, not a benchmark; the per-call times were not retained, so no paired SE beyond the per-call quantile is available.

C Where folding pays off, and equality totals

Table 2: Folding pays off only when every learner is constant; then one-row prediction is three to four orders of magnitude faster, with bit-identical outputs. Speedup is ours over upstream ngboost 0.5.11 for one row (range over fits).

Fits	all-constant fits	one-row speedup	one-row ms, upstream / ours
grid, concrete	6 of 24	4,897× to 8,153×	not recorded
grid, yacht	10 of 24	4,742× to 10,069×	not recorded
grid, credit-g	6 of 24	964× to 1,842×	not recorded
tuned, Friedman without signal	6 of 6	1,808× to 2,215×	320–394 / 0.18
Bitwise equality of fold only, direct traversal only and ours: all 6,994 test rows of 90 fitted models.			

D Applying the method to your own model

The method needs nothing from the data; it reads only the fitted model. With the package, after any NGBoost fit:

```
import ngbfast
ngbfast.fold(model)                      # plan stored as model.fold_
theta = ngbfast.pred_param(model, X)  # same bits as model.pred_param(X)
ngbfast.diagnose(model, n_train=n)    # constant share, Lemma 2 bound
ngbfast.patch()                          # optional: every fit folds itself
```

To reimplement it for another additive ensemble of the form of Equation (1): (1) for each learner decide whether it is constant (one node, all leaves equal, or a constant regressor) and compute its scaled step s_{ik} with the same operation order as the prediction loop; (2) per output column, subtract the constant steps before the first input-dependent learner into start_k (Equation (3)) and keep every later step, constant or not, in round order, unmerged; (3) probe each skipped learner once on a NaN row and an infinity row and reject those inputs in the folded path if it did; (4) validate and cast the input once per call and walk the remaining trees without per-tree checks; (5) compare the folded output bit for bit with the original loop on held-out rows before serving it. Lemma 2 gives the check for step (1) in advance: if each round's rows fall below $\max(\text{min_samples_split}, 2L)$, every tree will be constant.

E Environment and reproduction

Python 3.12.3, ngboost 0.5.11, scikit-learn 1.9.1, numpy 2.5.4; one OpenMP thread per process; shared 8-core host, load average 7–20; tuned runs took 5,451 s wall time. Grid: learning rate 0.01, 500 rounds, 15 one-row repetitions per point. Tuned picks: concrete depth 5, leaf 1, minibatch 0.8, rate 0.0177, 119 rounds; yacht 4, 10, 0.8, 0.0944, 49; credit-g 1, 20, 0.5, 0.0316, 234. Held-out NLL of the tuned models: concrete 2.925, yacht -0.934, credit-g 0.459 nats. Code: repository `ngbfast`, commit `d8dda3e`; upstream patch `patches/ngboost-fold-constant-trees.patch`. Results: `python -m bench.summarize` after the per-data-set runs listed in `bench/results/SCOPE.md`; every number in this paper is a macro written by `paper/results/make_numbers.py` from `bench/results/`.

Figure 3 gives the absolute one-row latency per arm.

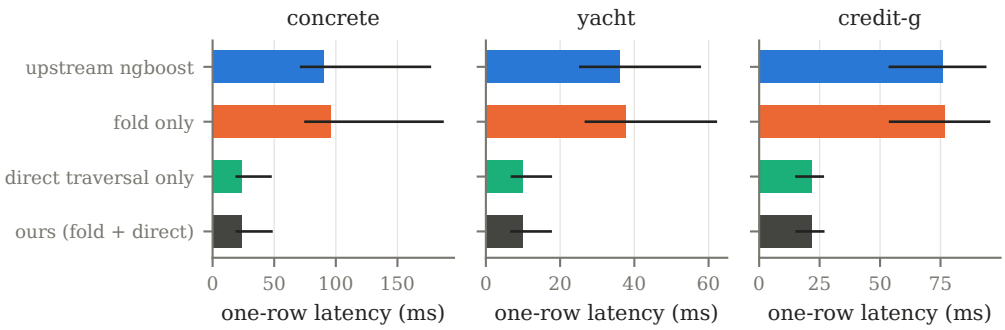


Figure 3: Direct traversal cuts upstream's one-row latency from 91.4 to 23.8 ms on concrete; fold only is no faster than upstream on tuned models. Bars: median over all repetitions and seeds; lines: 10th to 90th percentile.